Computer Olympic POSN KU Center (CAMP2 2568/2569)

State Space Search

Nopadon Juneam
Department of Computer Science
Kasetsart university

Outlines

- Basics of State Space Search
  - State Space Search with Breadth First Search (BFS)
  - State Space Search with Depth First Search (DFS)
  - Meet in the Middle (Bidirectional Search)

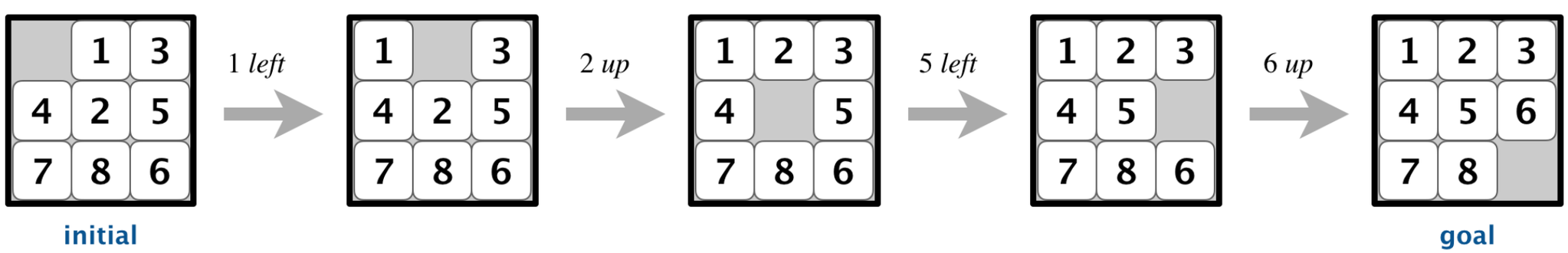
Practice Problem (Vjudge)

- UVa 11212 Editing a Book

State Space Search

- **State space search** can be described as a process in which *successive configurations* or *states* of an instance are considered, with the intention to find a *goal state* with the desired property from a given initial state.

- Problem are often modeled as a *state space*, that is, *a set of states a problem can be in*. The set of states forms a *graph* where two states are connected if there is an operation that can be performed to transform the first state into the second



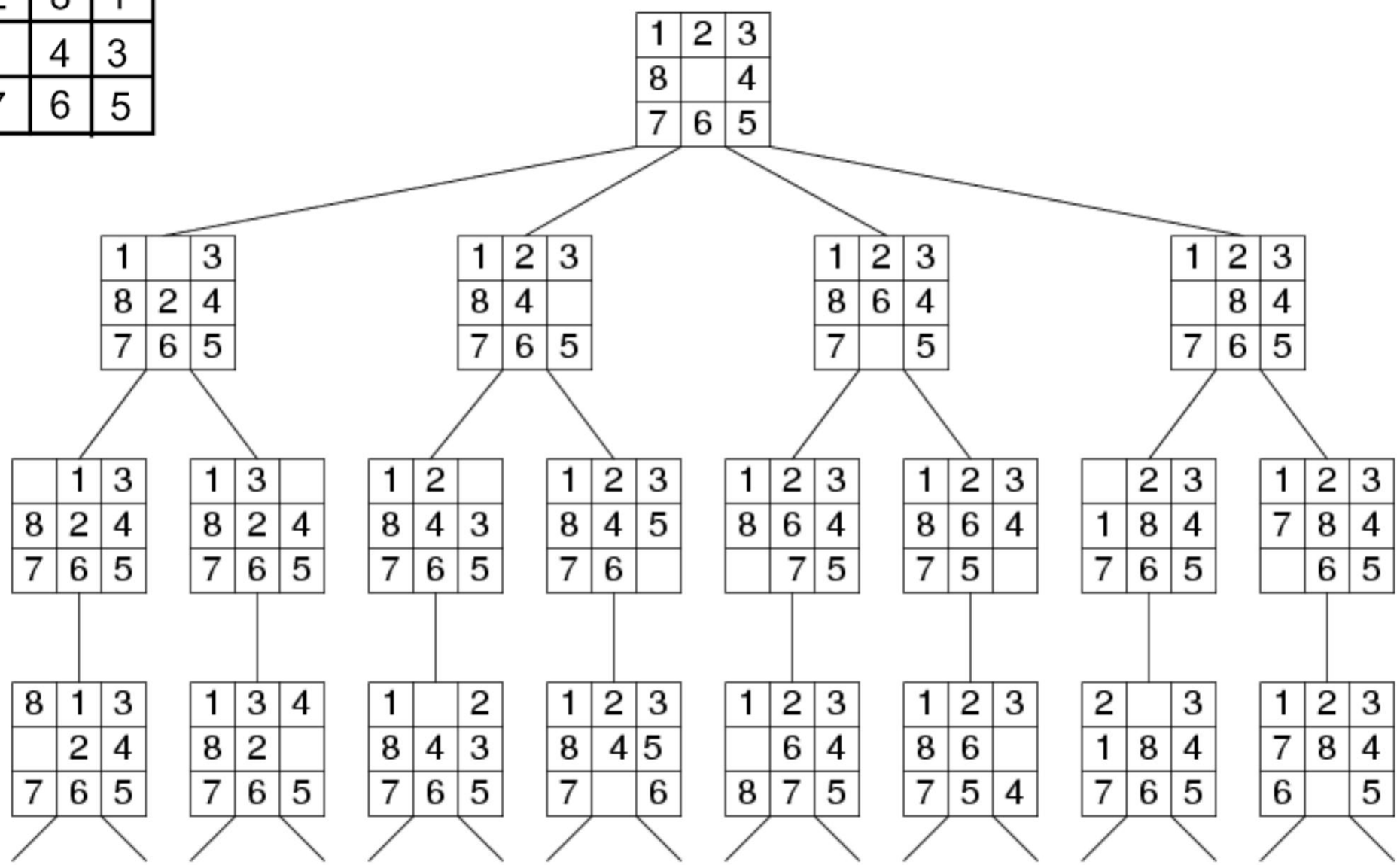
Example: 8 Puzzle

Initial State

1	2	3
8		4
7	6	5

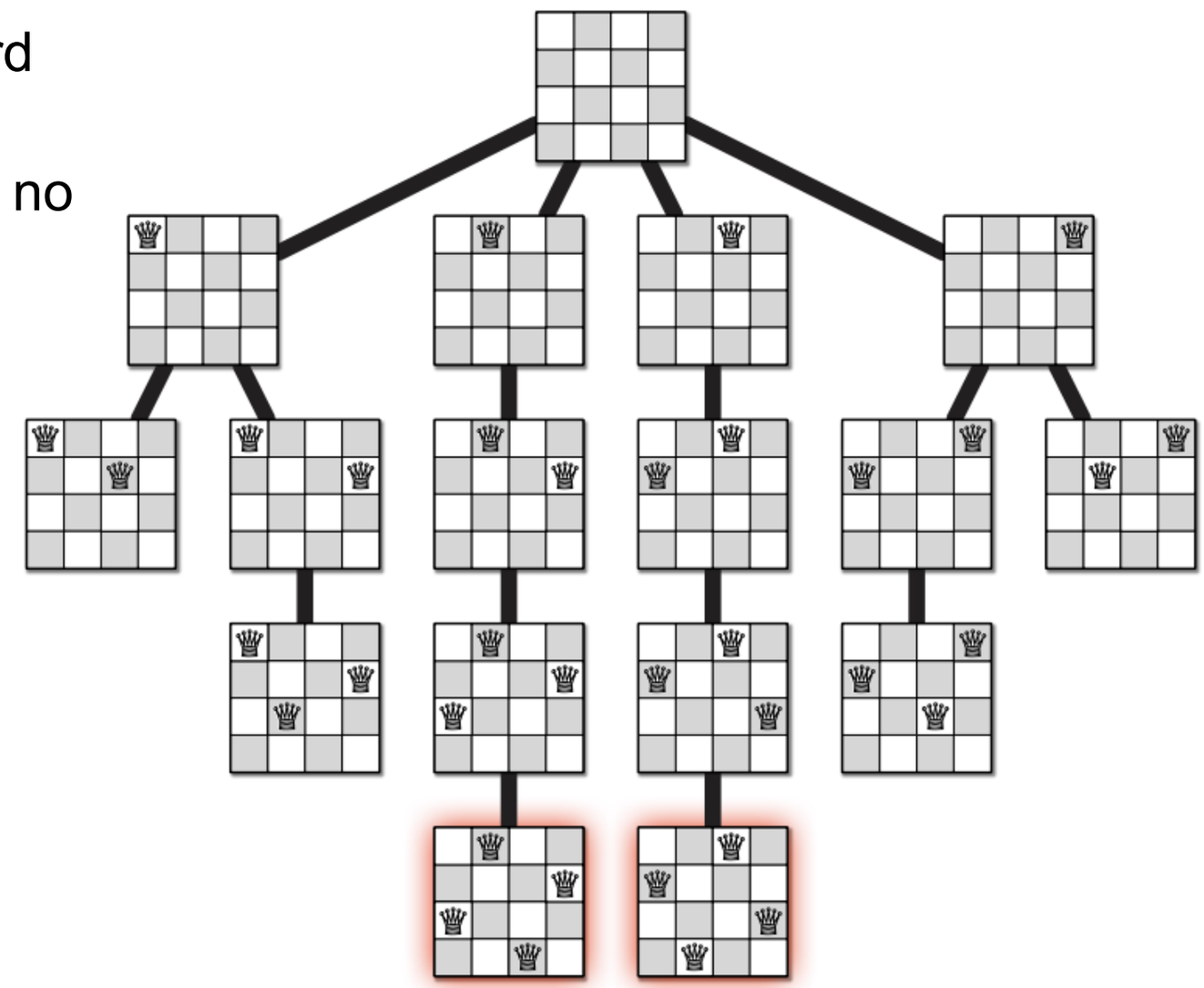
Goal State

2	8	1
	4	3
7	6	5



Example: 8 Queen

- **Initial state:** no queens on board
- **Goal state:** 8 queens on board, no queens attack each other



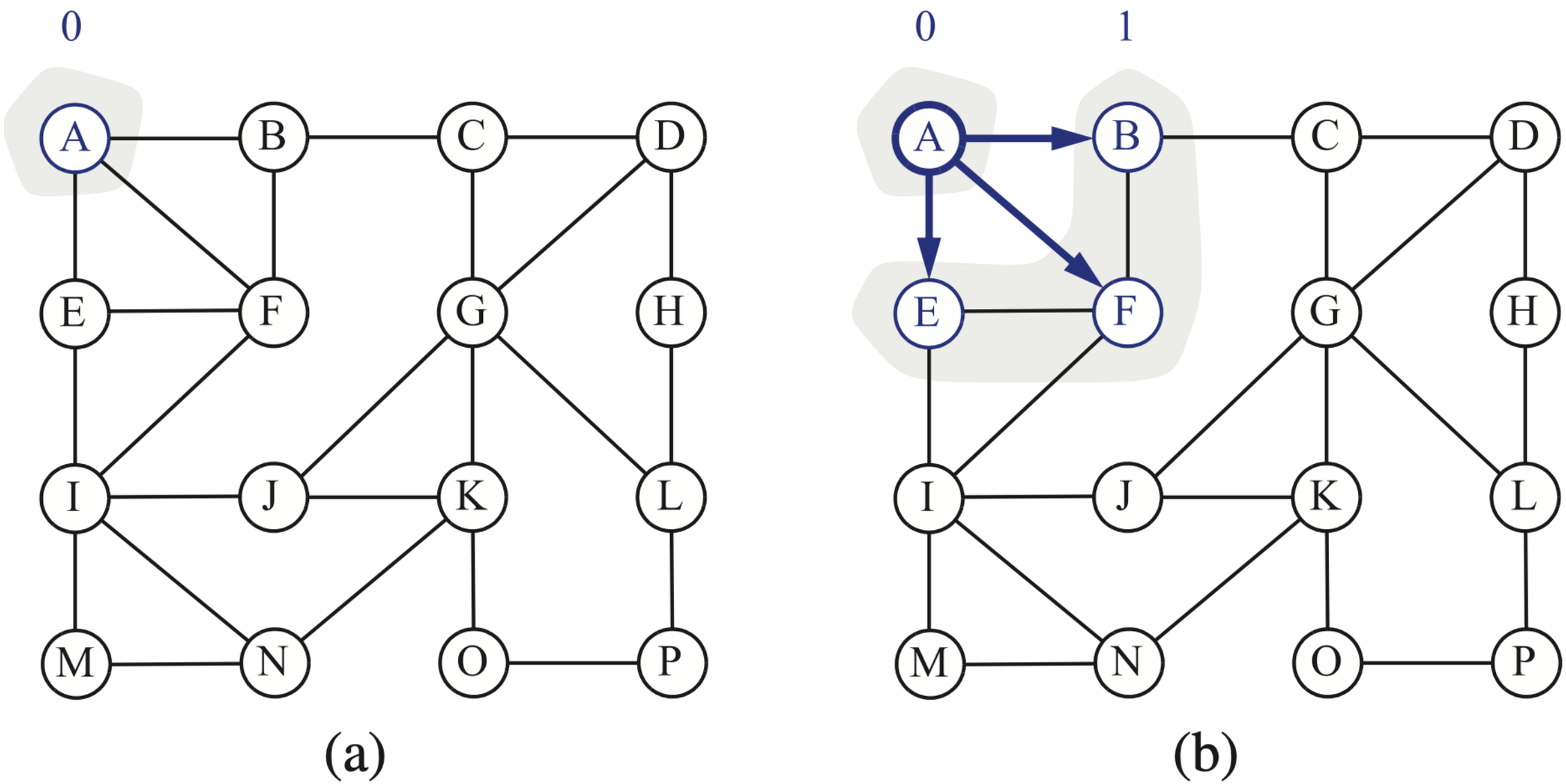
State Space Search vs Regular Graph Traversals

- State space search differs from *traditional computer science search methods (e.g. regular graph traversal)* because the state space graph is often implicit and is much too large to generate and store in memory. Instead, nodes are generated as they are explored, and typically discarded thereafter.

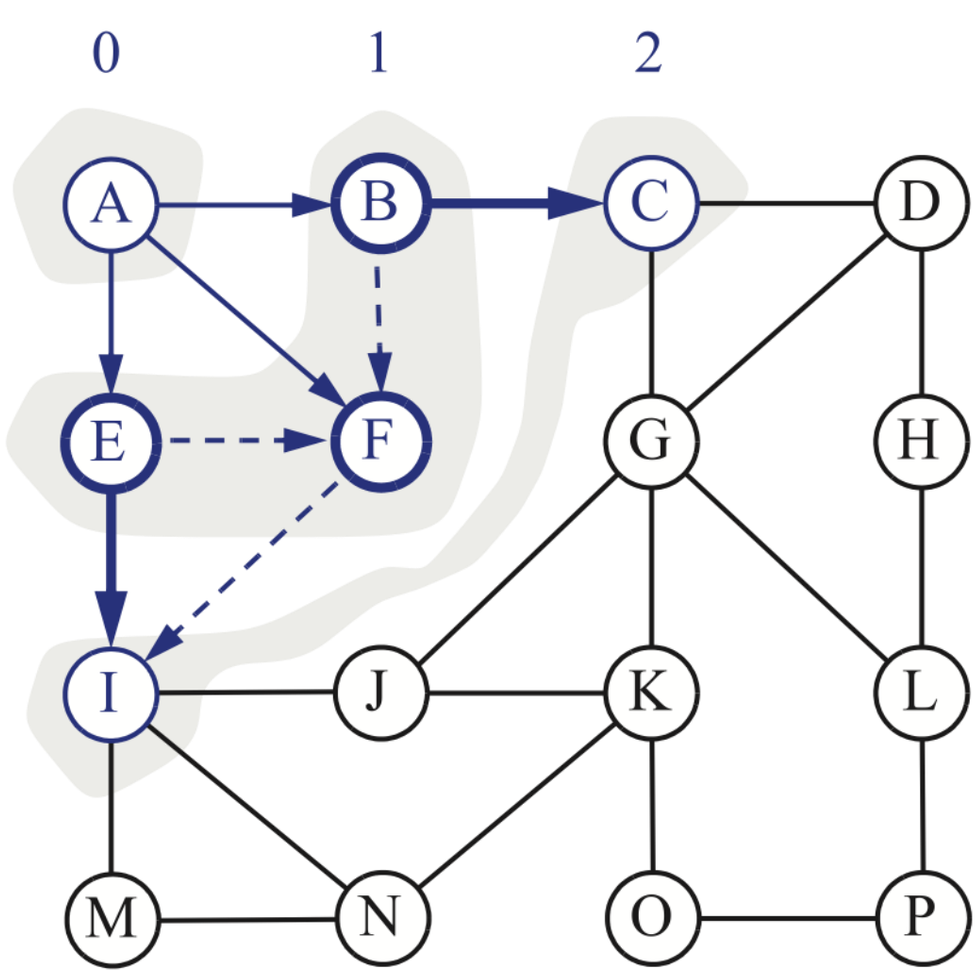
Search on State Space Graph

- Uninformed blind search
  - **State-Space Search with Breath First Search (BFS)**
    - Always expand the shallow unexpanded nodes first (always give a shortest path to the nodes)
    - Implementations need FIFO data structures (e.g. queue)
  - **State-Space Search with Depth First Search (DFS)**
    - Always expand the deepest unexpanded nodes first
    - Implementations need LIFO data structures (e.g. stack or recursion)

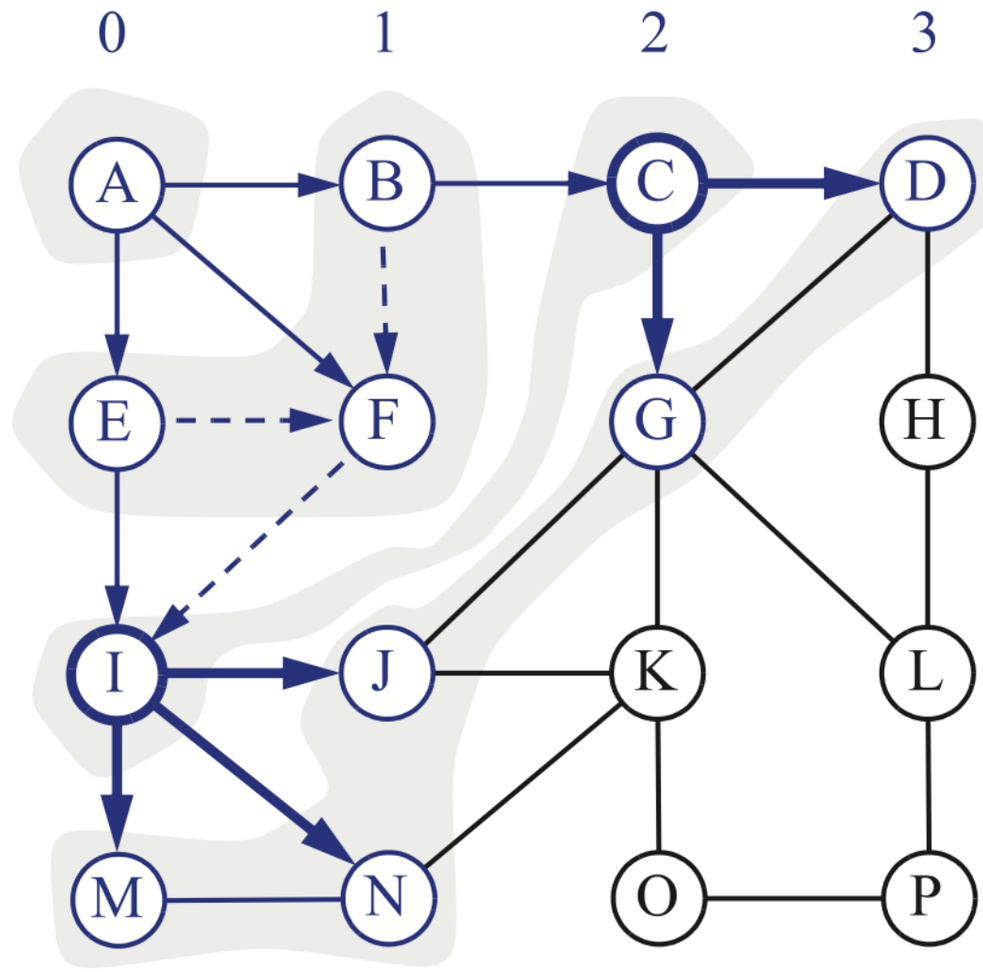
BFS Demo (1)



BFS Demo (2)

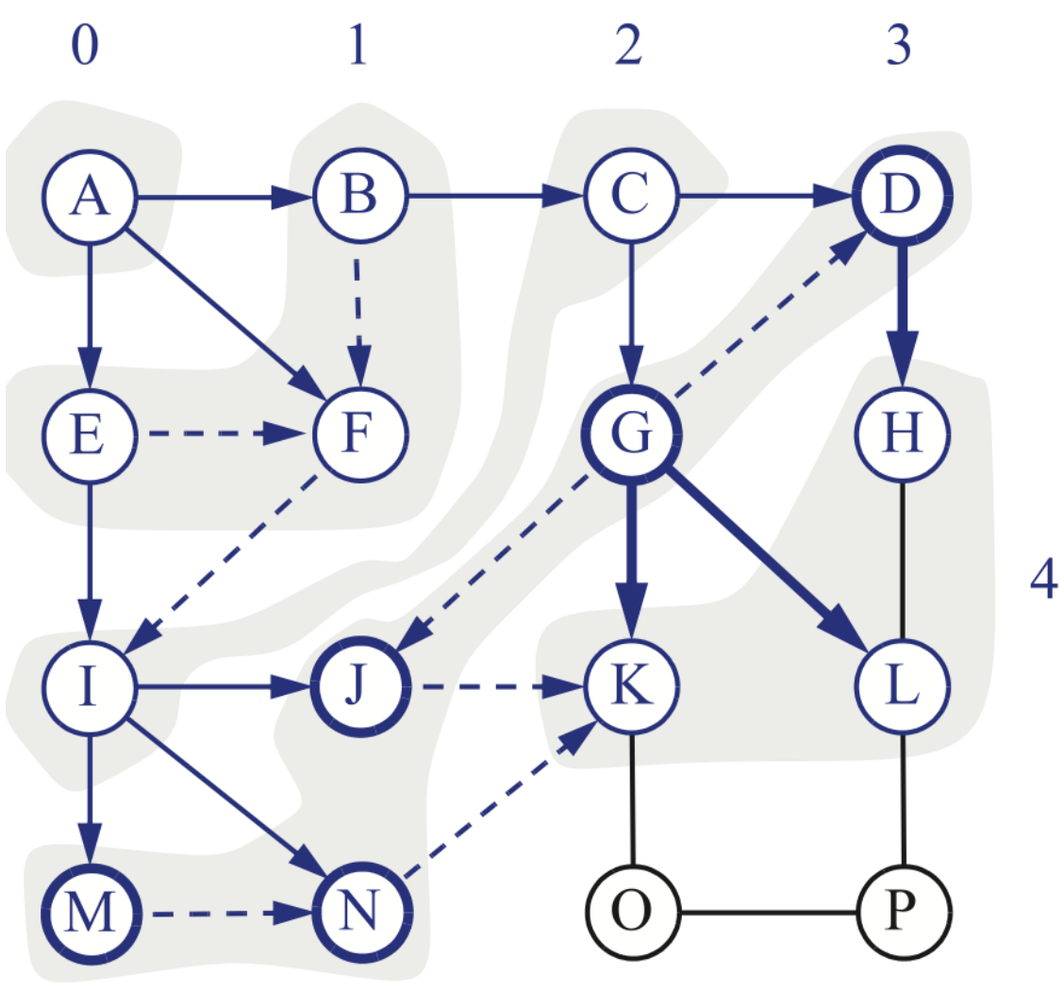


(c)

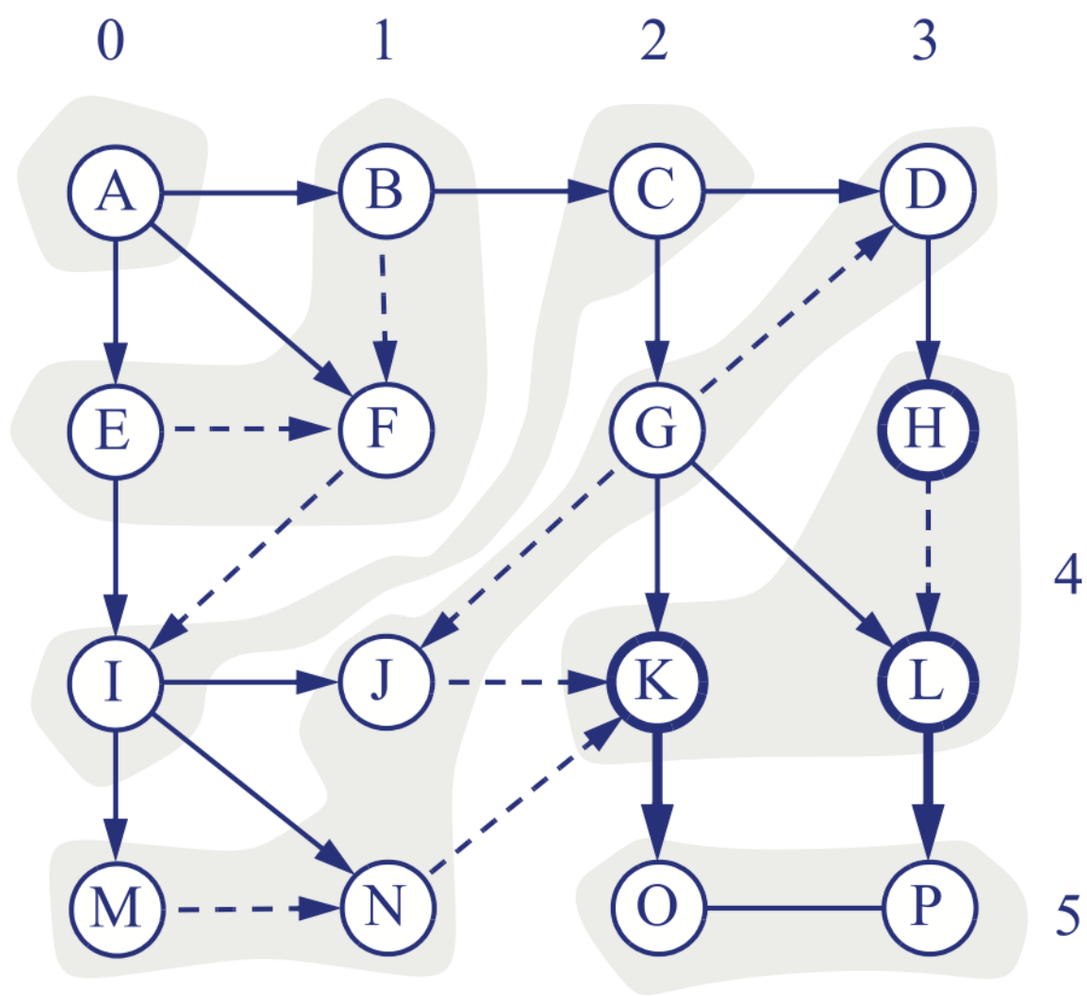


(d)

BFS Demo (3)

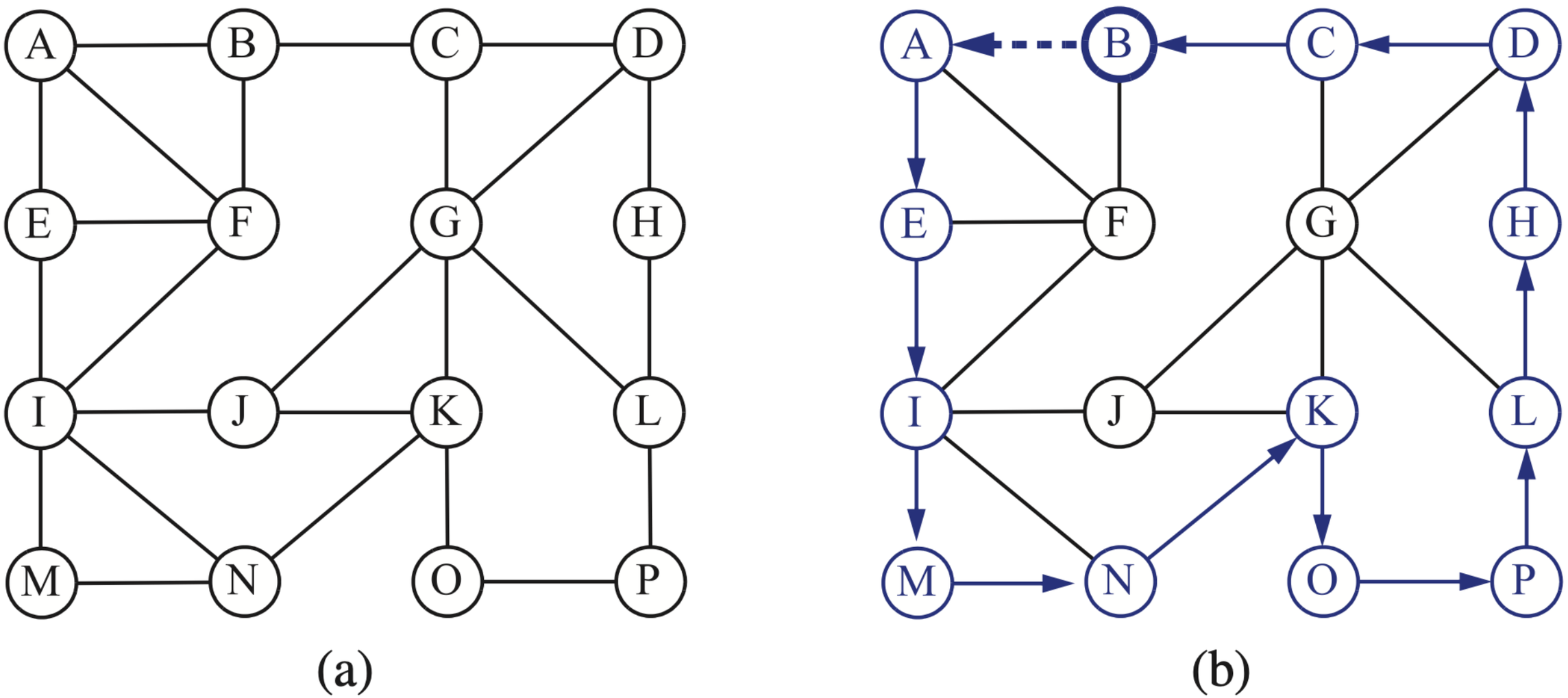


(e)

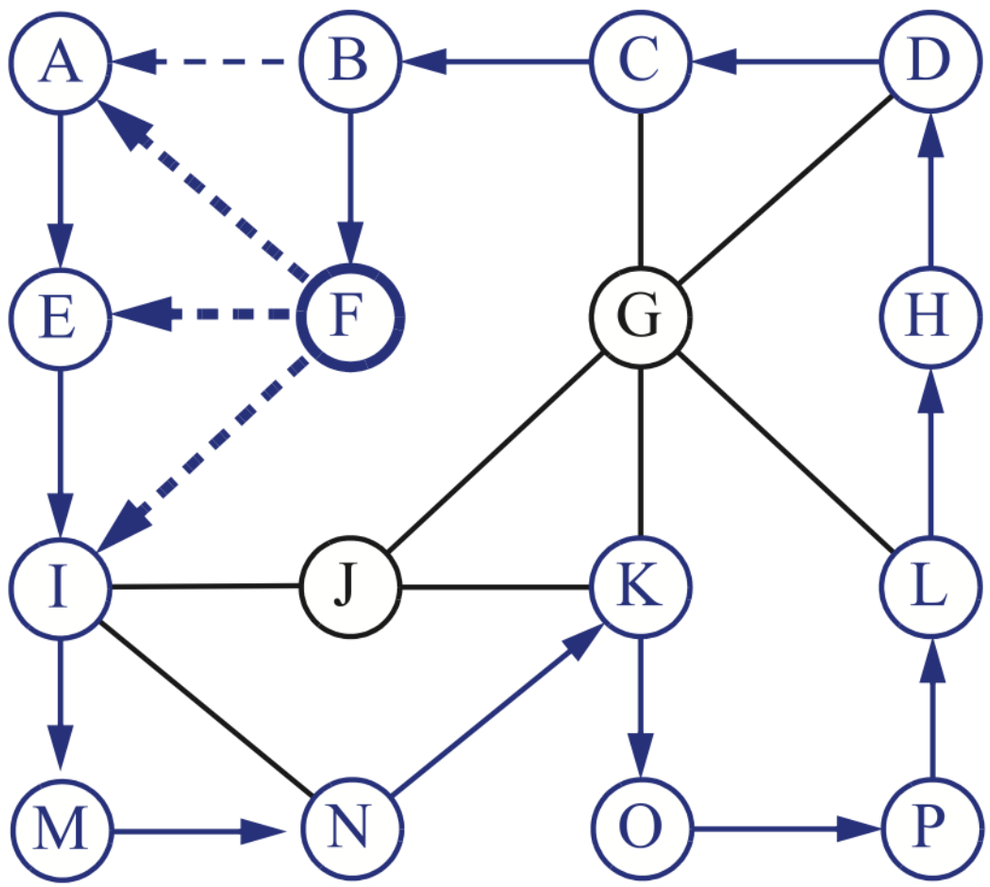


(f)

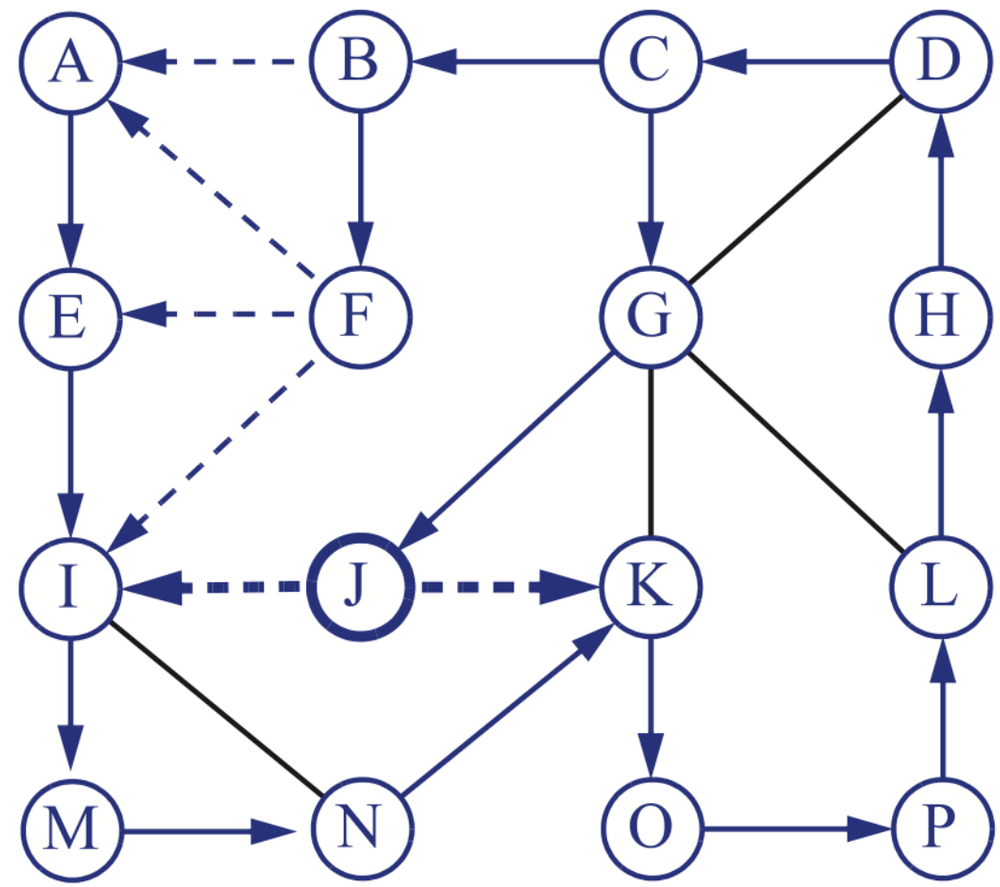
DFS Demo (1)



DFS Demo (2)

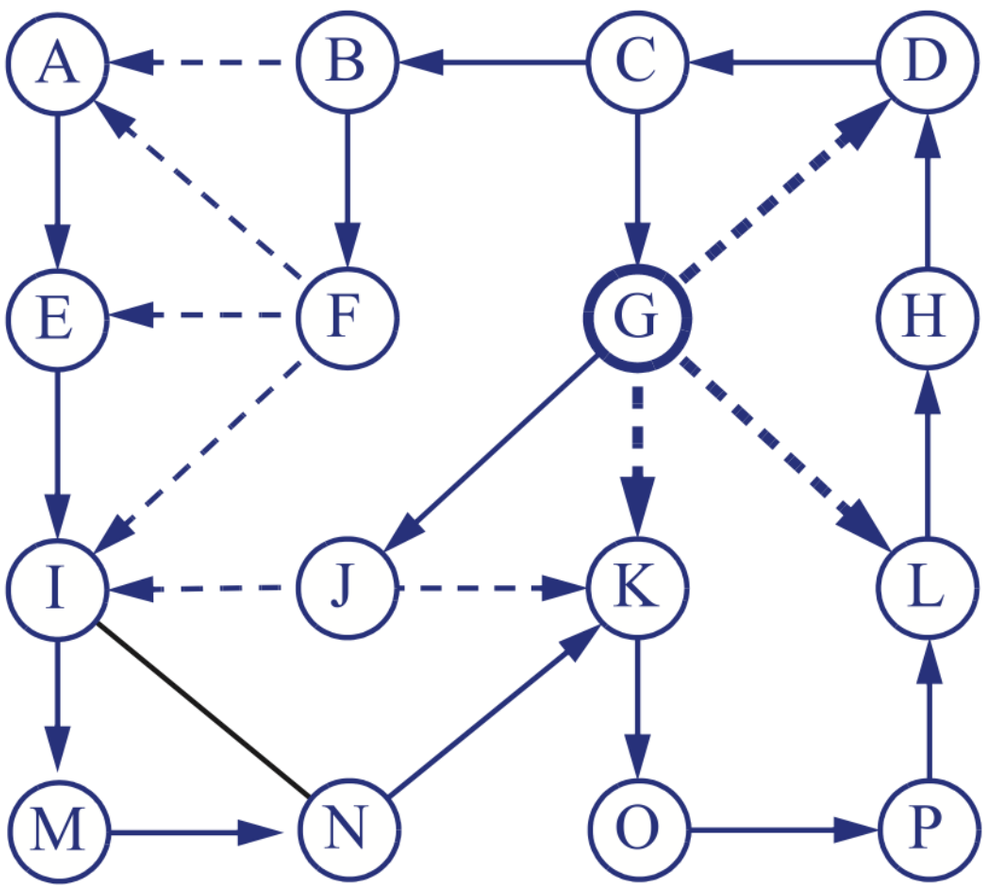


(c)

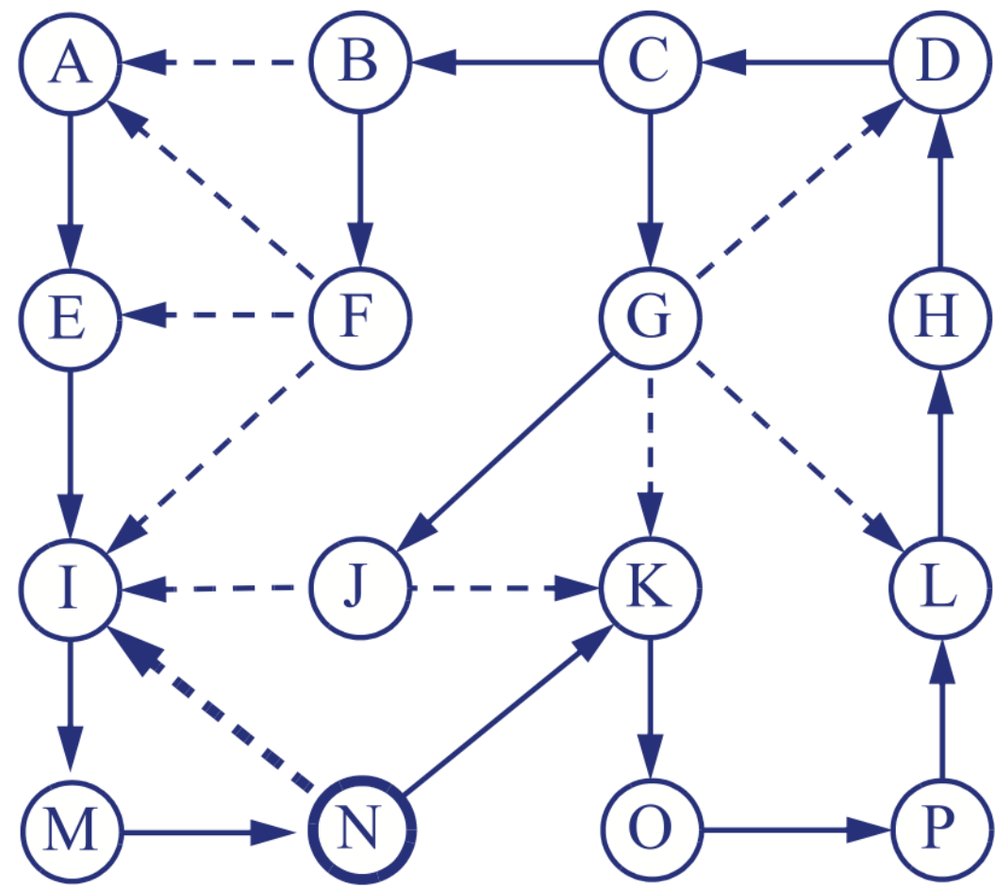


(d)

DFS Demo (3)



(e)



(f)

UVa 11212 – Editing a Book (1)

- <u>Problem statement</u>: Given n equal-length paragraphs numbered 1 to n, $1 \le n \le 9$, arrange them in the order of 1,2,…, n by using a clipboard

  - **Clipboard:**

    - We can press ctrl-X (cut) and ctrl-v (paste) several times.

    - We cannot cut twice before pasting, but can cut several contiguous paragraphs at the same time and they will be pasted in order

  The problem askes to compute the minimum number of steps required.

UVa 11212 – Editing a Book (2)

- **Example 1:**
  In order to make {2, 4, 1, 5, 3, 6} sorted,
  we can cut 1 and paste it before 2
  -> {1, 2, 4, 5, 3, 6}
  then cut 3 and paste it before 4
  -> {1, 2, 3, 4, 5, 6}

- **Example 2:**
  In order to make {3, 4, 5, 1, 2} sorted,
  we can cut {3, 4, 5} and paste it after {1, 2}
  -> {1, 2, 3, 4 ,5}

Trivial but Wrong Algorithm (1)

- **Loose upper bound**: A loose upper bound on the number of cut & paste operations would be n-1, where n is the number of paragraphs initially in the wrong positions
- Trivial algorithm (but may give the wrong answer):
  - Cut a paragraph in the wrong position, then paste it in the correct position.
  - After n-1 cut & paste operations, we have a sorted paragraph

Trivial but Wrong Algorithm (2)

- Trivial algorithm (but may give the wrong answer):
  - Cut a paragraph in the wrong position, then paste it in the correct position.
  - After n-1 cut & paste operations, we have a sorted paragraph.
- **Examples**
  {(3), 2, 1} -> {(2), 1, 3} -> {1, 2, 3}
  Done in 2 steps, this is at the minimum.

  {(5), 4, 3, 2, 1} -> {(4), 3, 2, 1, 5} -> {(3), 2, 1, 4, 5} ->
  {(2), 1, 3, 4, 5} -> {1, 2, 3, 4, 5}
  Done in 4 steps, but we can do in 3 steps

  {5, 4, (3, 2), 1} -> {3, (2, 5), 4, 1} -> {3, 4, (1, 2), 5} -> {1, 2, 3, 4, 5}

Analysis of the Problem

- Here is some analysis for the problem
  - There are at most n! permutations of paragraphs.
    - With n = 9, 9! = 362880. This is the number of nodes in our implicit graph
  - Given a permutation of length *n* (a node in the graph)
    - There are $\binom{n}{2}$ possible cutting points. There are n possible pasting points
    - Therefore, for each node, there are about $O(n^3)$ branches
  - Thus, the number of edges in the implicit graph will be $O(n! \cdot n^3)$
    - With n = 9, $9! \cdot 93 \approx 265M$ (too many edges to be stored in memory)

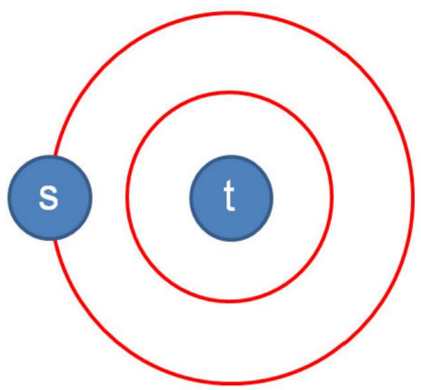
Possible Solution (1)

- If we run BFS on the largest test case with n = 9 from the goal state (i.e., the sorted permutation {1,2, …, 9}) to reach all other states, we will find that the maximum depth due to BFS is 5 (of course, we need to test it to see if this is true!)

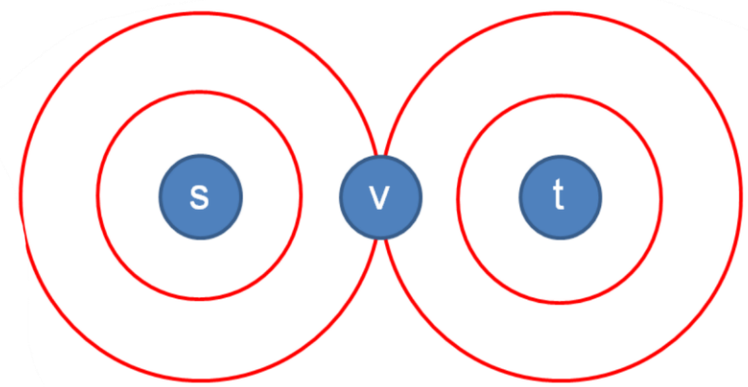
- **Meet in the middle:** The information above allow us to perform bidirectional search using BFS. That is, one search starts from the initial state (with max depth = 2), and the other starts from the goal state (also with max depth = 2)

Possible Solution (2)

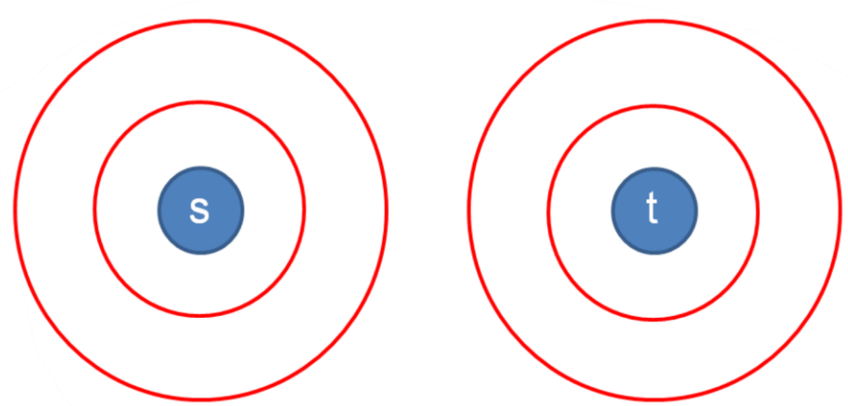
- With the bidirectional search, there are three possible cases to consider



Case 1: Example when s is two steps away from t



Case 2: Example when s is four steps away from t



Case 3: Example when s is five steps away from t

Analysis of Bidirectional Search

- Given a permutation of length n (the goal state), there are $O(n^3)$ branches in the implicit graph. If we run each BFS with max depth 2, the search requires just $O((n^3)^2) = O(n^6)$ operations per BFS
- With $n = 9$, $n^6 = 531441$ operations required!
- As the goal state remains unchanged throughout the search, we can compute the first BFS from the goal state t just once
- Then, we can compute the second BFS from the initial state upon query

Homework Problems (Vjudge)

- UVa-0167 The Sultan Successor
- UVa-0539 The Setters
- UVa-0677 All Walks of length “n”
- UVa-12346 Water Gate Management
- UVa-12348 Fun Coloring
- Uva-0321 The New Villa